ACTIVITY 1: PORTFOLIO CREATION USING AZURE

Azure Portfolio Launchpad

Personal & Professional Portfolio with Cloud Deployment

Submitted By

Prajwal M M – 25MCAR0148

Technologies Used

HTML5 • CSS3 • JavaScript • Azure Static Web Apps • Azure Functions • GitHub • GitHub Actions

Cloud Platform - Microsoft Azure

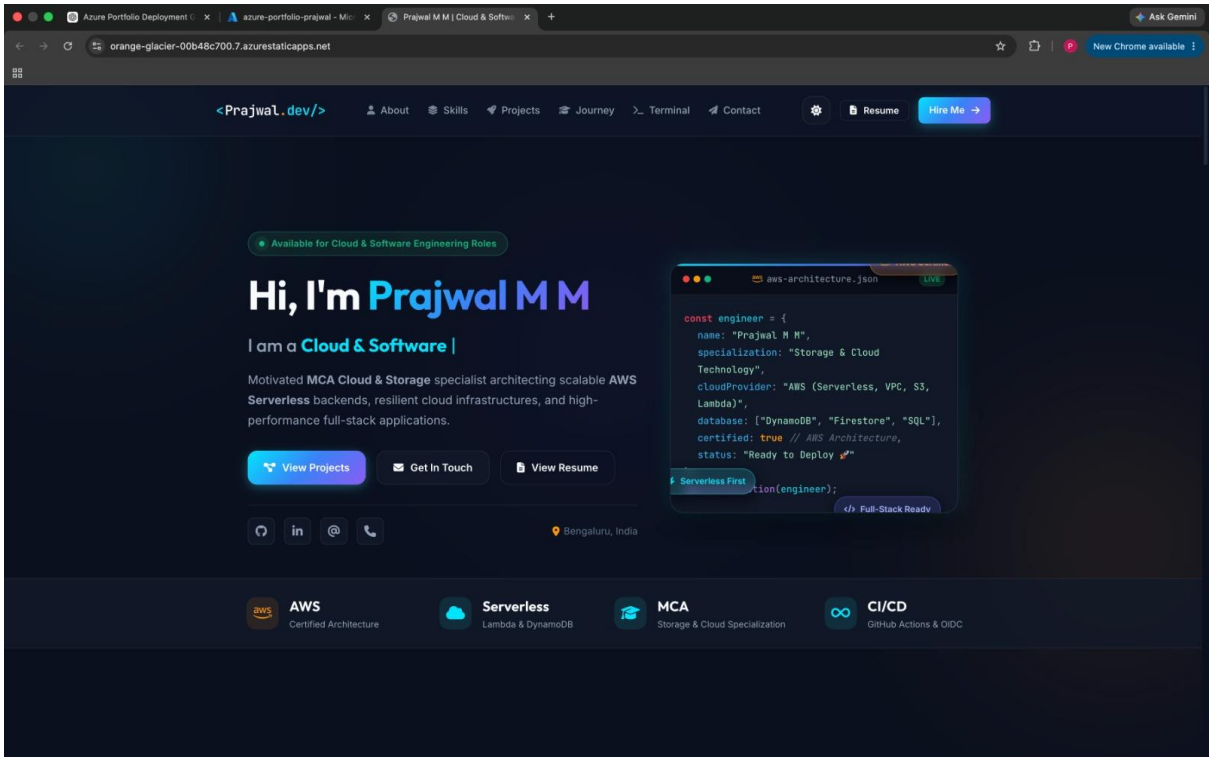
GitHub Repository

https://github.com/prajwalmm-20/Azure-portfolio

Live Portfolio

https://orange-glacier-00b48c700.azurestaticapps.net

Activity: 1 — Portfolio Creation



1. Introduction

The objective of this project was to design and develop a personal and professional portfolio website and deploy it using Microsoft Azure. The portfolio provides a centralized platform to present my profile, education, technical skills, projects, career journey, resume, and contact information.

The website was developed using standard web technologies including HTML5, CSS3, and JavaScript. To extend the portfolio beyond a static website, a serverless backend was implemented using Azure Functions. The contact form communicates with the Azure Function through an HTTP API.

The completed application is hosted using Azure Static Web Apps and integrated with GitHub Actions for automated deployment. This provides a simple cloud-based CI/CD workflow where changes pushed to the GitHub main branch are automatically deployed to Azure.

1.1 Project Objectives

The main objectives of the project were:

- To create a professional personal portfolio website.
- To present personal information, education, skills, projects, resume, and contact details.
- To design a responsive and user-friendly frontend.
- To deploy the portfolio using Microsoft Azure.
- To implement backend functionality using a serverless Azure Function.
- To connect the frontend contact form with a backend API.
- To maintain the complete project using Git and GitHub.
- To implement automated deployment using GitHub Actions.
- To verify backend activity using Azure monitoring metrics.
- To use a cost-conscious cloud architecture suitable for an Azure for Students subscription.

1.2 Project Scope

The project covers both frontend and backend components.

Frontend

The frontend provides the visual interface of the portfolio and includes:

- Home/Profile
- About
- Education
- Technical Skills
- Projects

- Career/Journey
- Resume
- Contact

Backend

The backend provides functionality for the contact form using an HTTP-triggered Azure Function.

2. Design and Development Approach

The portfolio was designed as a modern, responsive personal website with a focus on simplicity, readability, and easy navigation. The design uses a dark-themed visual style with clearly separated sections to present professional information in an organized manner.

The development was divided into three main areas:

1. Frontend development
2. Client-side functionality
3. Serverless backend integration

2.1 Portfolio Design

The website follows a structured single-page portfolio layout. A navigation menu allows users to quickly access different sections of the portfolio.

The major sections include:

- **Home/Profile** – Introduction and professional overview.
- **About** – Personal and professional information.
- **Education** – Academic background.
- **Skills** – Technical skills and areas of expertise.
- **Projects** – Selected technical and cloud projects.
- **Journey** – Career and learning progression.
- **Resume** – Resume access.
- **Contact** – Contact information and interactive contact form.

The interface was designed to be responsive so that the content remains accessible across different screen sizes and devices.

2.2 Frontend Development

HTML5

HTML5 was used to create the basic structure and content of the portfolio.

CSS3

CSS3 was used to control the visual presentation of the website.

JavaScript

JavaScript was used to provide interactive functionality.

2.3 Contact Form Validation

The contact form includes client-side validation before sending information to the backend.

The form checks:

- Name is provided.
- Email address is provided.
- Email format is valid.
- Subject is provided.
- Message is provided.
- Message contains a minimum number of characters.

If invalid information is entered, an appropriate validation message is displayed to the user.

After successful validation, the form sends the data to the Azure Function using an HTTP POST request.

2.4 Development Environment

The portfolio was developed using Visual Studio Code.

Git was used for version control and the project was maintained in a GitHub repository.

3. Technologies, Tools and Platforms

The portfolio uses a combination of frontend technologies, cloud services, development tools, version control, and CI/CD technologies. The selected technologies were chosen to keep the application simple, maintainable, serverless, and cost-effective.

3.1 Technologies Used

Technology / Tool	Purpose	Usage in Project
HTML5	Frontend structure	Creates the structure and content of the portfolio
CSS3	Styling	Responsive layout, dark theme, animations and visual design
JavaScript	Client-side functionality	Form validation, interactions and API communication

Technology / Tool	Purpose	Usage in Project
Node.js	Backend runtime	Runtime environment for the Azure Function
Azure Static Web Apps	Cloud hosting	Hosts the portfolio frontend
Azure Functions	Serverless backend	Processes contact form API requests
Git	Version control	Tracks project changes
GitHub	Source-code repository	Stores and maintains the complete project
GitHub Actions	CI/CD	Automatically deploys changes to Azure
Azure Monitor Metrics	Monitoring	Verifies Azure Function activity
Visual Studio Code	Development environment	Used to develop and manage the project

3.3 Why These Technologies Were Selected

Azure Static Web Apps

Azure Static Web Apps was selected because the portfolio primarily consists of static HTML, CSS, and JavaScript files. It provides a simple way to host the frontend and integrate it with a serverless backend.

Azure Functions

Azure Functions was selected to implement backend functionality without requiring a continuously running server or virtual machine. The contact form uses an HTTP-triggered function.

GitHub

GitHub provides source-code management and makes the complete portfolio project accessible as required by the activity.

GitHub Actions

GitHub Actions automates deployment. This eliminates the need to manually upload files to Azure whenever the portfolio is updated.

Visual Studio Code

Visual Studio Code was used as the primary development environment because it provides integrated Git support, extensions for Azure services, and an efficient environment for HTML, CSS, JavaScript, and Azure Functions development.

4. Azure Deployment

The portfolio was deployed on Microsoft Azure using Azure Static Web Apps. An Azure for Students subscription was used to keep the project cost-effective.

4.1 Azure Account

An Azure for Students subscription was created and activated. The subscription provided Azure credits that could be used for the project.

4.2 Resource Group

A resource group named:

rg-portfolio

was created to organize the Azure resources used for the portfolio.

4.3 Static Web App

An Azure Static Web App named:

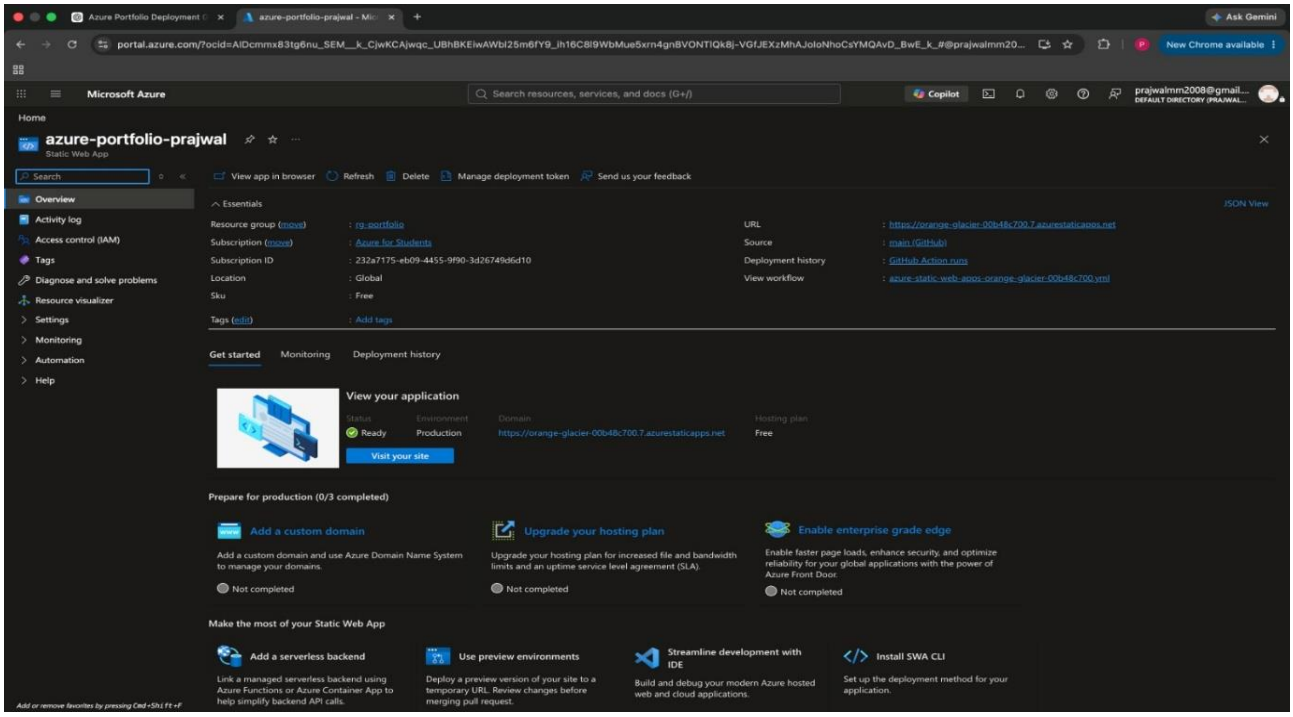
azure-portfolio-prajwal

was created using the Free plan.

The application was connected to the GitHub repository:

prajwalmm-20/Azure-portfolio

The main branch was selected for deployment.



4.4 Deployment

Azure automatically created a GitHub Actions workflow for the project.

Whenever changes are pushed to GitHub, the workflow automatically deploys the latest version of the portfolio to Azure.

5. Backend Implementation

To provide backend functionality, an Azure Function was added to the portfolio.

The function is used by the contact form and is accessed through:

/api/contact

When a visitor submits the contact form, JavaScript sends the form information to the Azure Function using a POST request.

5.1 Backend Testing

The contact form was tested using sample information.

Name: Test User

Email: test@example.com

Subject: Azure Backend Test

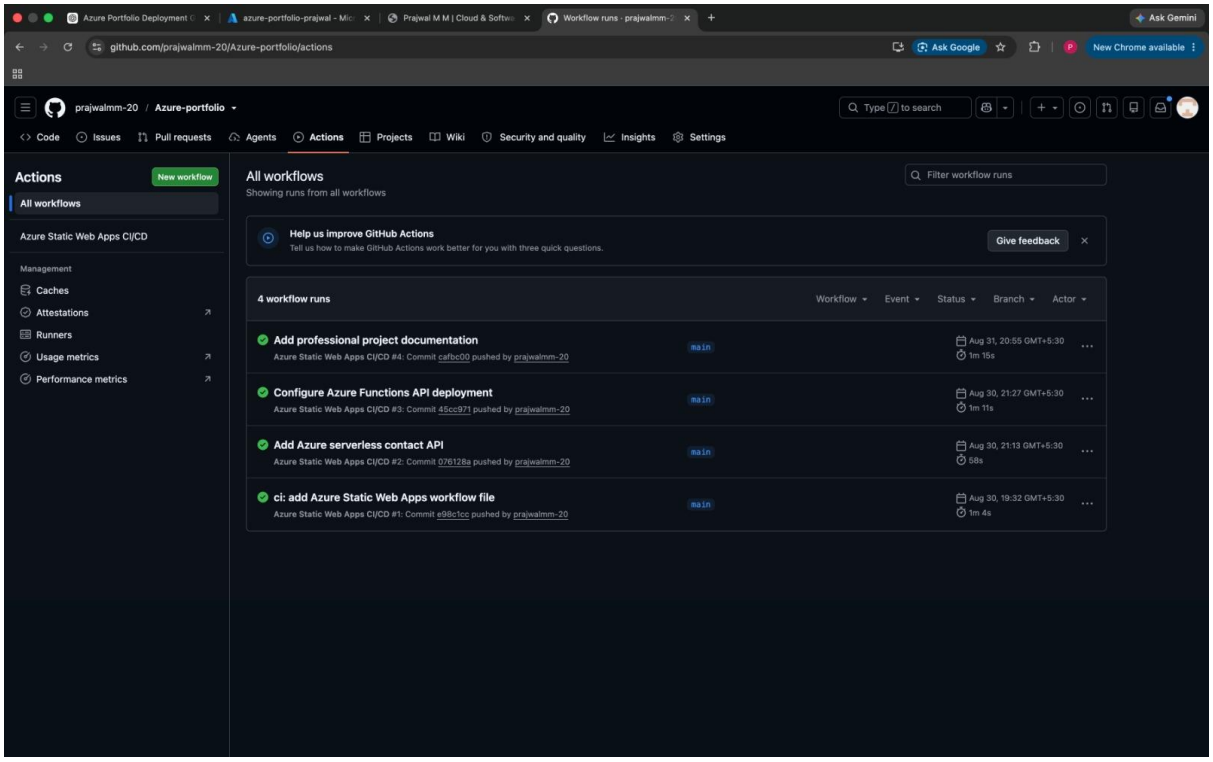
Message: Testing my Azure serverless contact API.

The test was successful and the expected response was displayed on the live website.

6. CI/CD and Monitoring

6.1 GitHub Actions

GitHub Actions was used to automatically deploy the portfolio to Azure.



6.2 Azure Monitoring

Azure Portal Metrics were used to verify the backend.

The FunctionHits metric recorded the Azure Function invocation after the contact form was submitted.

The test produced:

FunctionHits = 1

This confirms that the contact form successfully communicated with the Azure Function.

7. Results and Conclusion

7.1 Project Results

The portfolio was successfully developed and deployed using Microsoft Azure.

The following requirements were completed:

- Personal portfolio website created.
- Portfolio hosted on Azure.
- GitHub repository created and maintained.
- GitHub Actions CI/CD implemented.
- Azure Function backend implemented.
- Contact form connected to the backend.
- Live deployment successfully tested.
- Azure Function activity verified using FunctionHits.

7.2 Cost Optimization

The project was designed to minimize cloud costs.

The Free plan of Azure Static Web Apps was used for hosting, and a serverless Azure Function was used instead of a continuously running server or virtual machine.

This makes the architecture suitable for an Azure for Students subscription.

7.3 Conclusion

The project successfully demonstrates the development and deployment of a professional portfolio using Microsoft Azure.

The final application combines a responsive frontend, serverless backend, GitHub version control, automated CI/CD, and Azure monitoring.

The project provided practical experience with cloud hosting, serverless computing, APIs, GitHub Actions, and Azure deployment.